Feature: Calendar pick

From build-plan: feature 12h

Size: light - one piece of logic (which of a person's Google calendars count as busy); the rest is wiring: one more Google permission, one column, and the calendar card moved onto Settings.

Branch: feature/12h-calendar-pick

Status: verified 2026-10-12: 12h.1 built, shared 188 and backend 1010 tests pass, the frontend builds and lints; the check against a real Google calendar moved to 10b (Amendments). Then the final review of the whole feature.

Goal

A person connects their Google calendar from Settings, and picks which of their Google calendars count as busy. Today only the main one is read (feature 3), so an appointment kept in a second calendar ("Work", "Family") is offered to customers as free. After this, every ticked calendar blocks the booking window, and the pick is theirs.

In scope

- **Calendar page**: `/settings/calendar`, "Calendar" last in the Settings section list. It holds the connection card as it is today (Connect, Disconnect, Reconnect, the outcome line), moved off the dashboard home, and under it, once connected, the list of the person's Google calendars, each a tick box, with one Save. Google sends the browser back to this page after a connect.
- **Your own calendar only**, as today: any member with a person on the business connects and picks for themselves; the business and the person always come from the session.
- **The permission**: the consent asks for `calendar.calendarlist.readonly` with the two it asks today. A new connect needs all three, or ends as `missing_permission`, its words changed to "leave every box ticked". A connection made before this has two: it keeps working on the main calendar, and the page says "Reconnect to choose which calendars count".
- **What counts**: the busy read asks Google for every ticked calendar in one call. Any ticked calendar Google cannot read makes the person's times unreadable, as one unreadable calendar does today (never read as free). None ticked reads nothing from Google.
- **The pick's life**: a new connection starts with the main calendar ticked (Decisions, 1). A reconnect with the same Google account keeps the pick; with another account it starts again at the main one. Disconnect deletes it with the connection.
- **A ticked calendar that left the account** (deleted, or unsubscribed at Google) shows on the list as "no longer in your Google account", ticked, so the owner can untick it and save.

Out of scope

- Writing bookings into any calendar but the main one (unchanged since 5d).
- An owner picking for another person; other providers (Microsoft, Apple).
- The in-app calendars (12b) and opening hours for the public.
- Google's app verification for production (every calendar permission needs it; 10b).

Build loop

Light: one step, one plan yes. After the yes it runs straight through: build, tests and checks, tick the box, write the build log entry and commit it to buildlogs, commit and push to `feature/12h-calendar-pick`, report. No step review; the review comes once, at `/complete`, which merges on Frank's yes.

Build steps

- [x] **12h.1 Calendar pick.** "Summit's admin opens Settings, Calendar, connects Google, ticks 'Work' as well as the main calendar and saves: an event in 'Work' tomorrow at 10:00 disappears from Summit's booking window. Unticking it brings 10:00 back. Disconnecting removes the pick."
  - Shared: migration adding `busyCalendarIds text[] not null default '{primary}'` to `calendar_connection` (existing rows keep reading the main calendar); the save's schema.
  - Backend: the third scope and the renamed scope check; the seam gains `listCalendars` and `findBusyBlocks` takes the ids; `getBusyTimes` passes the connection's pick; the save keeps or resets the pick by account; two routes (Contracts); the callback lands on `/settings/calendar`.
  - Frontend: the Calendar page and section entry, the card moved from the dashboard home, the pick list with its Save.
  - Check in the build whether Google shows the new permission for the OAuth client as it is; if it refuses, the scope is added to the client's Data Access in Google Cloud, which is Frank's.

Done when: the tests below pass, the frontend builds and lints, and by hand on localhost the scene above works with Frank's own Google account (he presses Connect and passes Google's consent; the event is read with `calendar:check`).

Files / areas

- `packages/shared/db/booking-tables/calendar-connection-table.ts`, a new migration, `packages/shared/zod-validation/` (the save's schema), `packages/shared/calendar/`
- `backend/lib/calendar/` (`google-oauth-client.ts`, `calendar-provider.ts`, `google-calendar-provider.ts`, `get-busy-times.ts`, `get-fresh-access-token.ts`, `save-calendar-`

`connection.ts`, `finish-google-connect.ts`, a new `list-calendars` and `save-busy-calendars`), `backend/routes/calendar-routes.ts` and its test
- `frontend/app/(dashboard)/settings/calendar/page.tsx`, `frontend/components/settings/` (section list, the calendar screen), `frontend/components/calendar/calendar-connection-card.tsx`, `frontend/components/dashboard/dashboard-home-screen.tsx`, `frontend/lib/api-client/calendar/`

Data / contracts

Both routes sit beside `/calendar/connection`, under the same dashboard CORS, CSRF, no-store and `booking` module middleware; any member, own person only (`409 no_person` otherwise, as today).

`GET /calendar/calendars` `200 { calendars: { id, name, isMain, counts }[], missing: string[] }`: Google's calendar list, the main one first with id `primary`, `counts` true when ticked; `missing` the ticked ids no longer on the list. `409 no_connection`; `409 reconnect_needed` when the connection needs reconnecting or lacks the list permission; `503 calendar_unreachable` when Google does not answer. Names are rendered by React as text.

`PUT /calendar/busy-calendars` `{ calendarIds: string[] }`, strict: no repeats, at most 50 (Google's free/busy limit), each id 1 to 1024 characters. Every id must be on the person's Google list as read at the save, or already ticked (so a calendar that left can stay until unticked): `400 { error, field: "calendarIds" }` otherwise. `200` with the GET's answer. Same 409s and 503.

Stored: `calendar_connection.busyCalendarIds`, Google's ids, the main calendar as `primary`. Reset to `{primary}` when a reconnect brings another Google account.

Testing

`npm run test --workspace=@scheduleads-app/shared`, `npm run test --workspace=backend` (local Postgres, migrated and seeded), `npm run build` and `npm run lint` for the frontend. One test per rule that would hurt if broken:

- the busy read asks for every ticked calendar and joins their times; one unreadable calendar throws; none ticked asks Google nothing
- the consent asks for the three permissions; a connect without the list permission is `missing_permission` and saves nothing
- a reconnect with the same account keeps the pick, with another account starts at `primary`
- the list marks ticked and missing calendars; the save refuses an id not on the list and keeps a missing one already ticked; one person's save never touches another's connection
- the schema refuses repeats and more than 50

Decisions (made while planning, Frank's to overrule)

1. **A new connection starts with the main calendar ticked.** Connecting has meant "read my main calendar" since feature 3; starting with nothing would let customers book over the person's appointments from the moment they connect. 2. **One unreadable ticked calendar makes the person unreadable**, never free: a picked person shows "try again", "any available" leaves them out, as today. 3. **None ticked is allowed** (the client decides): the page says that then nothing in Google stops a customer booking. Bookings still go into the main calendar. 4. **Another Google account starts again at the main calendar**; its old ids mean nothing there.

Notes for the AI

- 12g's patterns for the page: a client component through `dashboardApiClient`, each error under its field, read-only nothing (every member edits their own).
- The card's `?calendar=` outcome handling moves with it unchanged.
- `calendar:check` follows the pick through `getBusyTimes`; no change there.

Amendments

- **2026-10-12, 12h.1's Done when:** the check by hand with Frank's own Google account could not run: Google's OAuth client still lists only the old `localhost:3001` callback, because the Oct 3 port move never reached Google, so Connect ends in `redirect_uri_mismatch`. Frank decided not to go to Google Cloud for it now: the new address, the list permission and this check go into 10b's one trip to Google Cloud. The step stands on its saved tests (Google faked) and the page checked on localhost without a connection.

Implementation walkthrough

Feature 12h was specced light, one step, and kept it: 12h.1 in `8a52737`, the checkpoint for the final review in `88292e5`, the review fixes in `5ba8870`. The first independent review passed with three P3 findings (F-407 to F-409), all fixed; the second, over the whole delta, closed them and left two P3s open (F-410, F-411), carried. The check by hand with a real Google account could not run: the Google OAuth client still lists only the old `localhost:3001` callback, because the Oct 3 port move never reached Google. Frank moved that check, the 3401 callback and the new permission into 10b's one trip to Google Cloud.

the pick (`packages/shared`)

Migration 0030 adds `calendar_connection.busyCalendarIds text[] not null default '{primary}'`, so every connection made before 12h keeps reading the main calendar. `busyCalendarsValidationSchema` is strict: at most 50 ids (Google's free/busy limit), no repeats, each 1 to 1024 characters.

Google (`backend/lib/calendar/`)

`google-oauth-client.ts` asks for a third permission, `calendar.calendarlist.readonly`; `hasEveryCalendarScope` requires all three for a new connect, and `canListCalendars` tells a connection made before 12h. The seam gains `listCalendars` (the person's list, at least free/busy readers, the main one first as `primary`, the name the person gave a shared calendar preferred, paged up to ten pages) and `findBusyBlocks` takes the ids and answers each calendar's blocks apart (F-409): the main calendar is found under its own key or as the one key left over, any missing or unreadable calendar throws, never reads as free.

the busy read

`getBusyTimes` reads the pick first and asks Google nothing, not even for a fresh key, when nothing is ticked (F-408). It takes `crossOffOnMain`: a moved booking's own span is crossed off the main calendar's blocks only, the one that holds its event; `find-free-times.ts` passes it instead of crossing off the joined list (F-409). `save-calendar-connection.ts` keeps the pick when the same Google account reconnects (case-insensitive, read against the row as it was) and starts again at `primary` for another account.

the routes (`routes/calendar-routes.ts`)

`GET /calendar/calendars` and `PUT /calendar/busy-calendars`, own person only, under the dashboard middleware. `find-calendar-choices.ts` reads the list live; `to-calendar-choices.ts` marks what counts and what left the account; `save-busy-calendars.ts` accepts only ids on the list or already ticked, and writes only the row as read (connection, business, person and account). Refusals: `409 no_connection`, `409 reconnect_needed` with two messages (a connection Google stopped accepting reads nothing; one made before 12h still reads the main calendar, F-407), `503 unavailable`, `400` on `calendarIds`. The callback now lands on `/settings/calendar`.

the page (`frontend/`)

`/settings/calendar`, "Calendar" last in the Settings list. The connection card moved off the dashboard home unchanged except its `missing_permission` words (every box, three now) and an `onChanged` that reloads the pick after a disconnect. `busy-calendars-part.tsx` shows the tick list with one Save, a calendar that left as "No longer in your Google account", and a warning when nothing is ticked.

carried

F-410 (nothing ticked with a lapsed connection is still told customers cannot book) and F-411 (a calendar hidden at Google shows as gone; the list asks without `showHidden`) are open P3s, both best settled at 10b against a real account.

Findings

12h/F-407 [P3] closed - The Calendar page says "only your main calendar counts" when the connection itself needs reconnecting, though then no calendar is read at all

File: frontend/components/settings/busy-calendars-part.tsx:39; backend/lib/calendar/find-calendar-choices.ts:44-45; frontend/lib/api-client/calendar/fetch-calendar-choices.ts:31 **Found:** 2026-10-12 by /audit independent (scope: current; lenses: quality, security, performance, tests) **Why it matters:** `reconnect_needed` from GET /calendar/calendars has two causes: a connection made before 12h (no list permission, still reading the main calendar), and a connection Google stopped accepting (`needs_reconnect`, where getFreshAccessToken throws and every busy read fails, so the person is "try again" or left out). The page answers both with "Until then only your main calendar counts.", which is true for the first and false for the second, under a card that is at the same moment saying Reconnect. **Suggested fix:** Give the two causes their own code or message (for example keep `reconnect_needed` for the broken connection and add a distinct one for "connected before the list"), and show "Until then only your main calendar counts" only for the second. **Resolution:** Fixed 2026-10-12: the two causes are apart. A connection made before 12h answers "Reconnect … Until then only your main calendar counts."; one Google stopped accepting answers "Reconnect your Google calendar. Until then none of your calendars is read, so customers cannot book you." (find-calendar-choices.ts `no_list_permission`, calendar-routes.ts refuseChoices); the page shows the API message as it is. Tests: both messages in calendar-routes.test.ts. Closed 2026-10-12 by independent review of 5ba8870: find-calendar-choices.ts returns `no_list_permission` only after getFreshAccessToken succeeded, so a needs_reconnect row (which throws first) always gets the "none of your calendars is read" message and a pre-12h row the "only your main calendar" one; calendar-routes.ts refuseChoices words them apart and both route tests assert them. One corner the two repairs together left is recorded as F-410.

12h/F-408 [P3] closed - With nothing ticked, a busy read still renews the Google key, and a connection Google stopped accepting still makes the person unbookable

File: backend/lib/calendar/get-busy-times.ts:20-22 **Found:** 2026-10-12 by /audit independent (scope: current; lenses: quality, security, performance, tests) **Why it matters:** The empty-pick return sits after getFreshAccessToken, which refreshes an expiring key at Google's token endpoint and throws CalendarReconnectNeededError on a `needs_reconnect` row. A person who chose that nothing in Google counts (Decision 3) is therefore still shown as "try again" or left out of "any available" whenever their Google permission lapses, and every free-times read for them can still cost a token call. The spec says none ticked "reads nothing from Google"; the test only counts free/busy calls, so it passes. Whether a lapsed connection should still block a person who picked nothing is a product call (the booking event write would fail too). **Suggested fix:** Frank's call. If "nothing ticked" should never block, read the row's `busyCalendarIds` (and status) before asking for a fresh key and return `[]` first; add a test with a `needs_reconnect` row and an empty pick. **Resolution:** Fixed 2026-10-12, keeping the spec ("None ticked reads nothing from Google", decision 3): getBusyTimes reads the pick first and returns [] before

asking for a fresh key. Test: nothing ticked with a needs_reconnect row and an expired key makes no token or free/busy call. Closed 2026-10-12 by independent review of 5ba8870: get-busy-times.ts:27-38 selects the row's busyCalendarIds and returns [] before getFreshAccessToken, and :41 re-checks the pick the key was read with; get-busy-times.test.ts asserts no token and no free/busy call for an empty pick on a needs_reconnect row with an expired key. No new defect in this file.

12h/F-409 [P3] closed - A moved booking's own span is now crossed off every ticked calendar, so an event in "Work" inside that span is read as free

File: backend/lib/calendar/google-calendar-provider.ts:73; backend/lib/scheduling/find-free-times.ts:137-143 **Found:** 2026-10-12 by /audit independent (scope: current; lenses: quality, security, performance, tests) **Why it matters:** When a booking moves, its own start to end is cut out of the person's Google busy blocks (7b decision 11), because the booking's event sits in the main calendar and free/busy cannot say which event is which. findBusyBlocks now flattens all ticked calendars into one list, so the cut also removes busy time from other calendars, which never hold the booking. Example: a 10:00-11:00 booking, a "Work" event at 10:00-11:00 added later; moving the booking to 10:30 is offered although Work says busy. Narrow (the new time must overlap the old span), and the main calendar had the same blind spot before, but for the other calendars it is now avoidable. **Suggested fix:** Keep which calendar each block came from (or return the main calendar's blocks apart) and cross the moved booking off the main calendar's blocks only; one find-free-times test with an overlapping event in a second calendar. **Resolution:** Fixed 2026-10-12: findBusyBlocks keeps each calendar apart (`CalendarBusyBlocksType` , the main one found under its own key or the one key left over), and getBusyTimes takes `crossOffOnMain` , crossing a moved booking off the main calendar only; find-free-times passes it. Test: the same hour busy in the main calendar and in Work, with the span crossed off, leaves Work busy. Closed 2026-10-12 by independent review of 5ba8870: google-calendar-provider.ts returns one entry per asked id in order and throws when any is missing or errored; get-busy-times.ts:52-54 crosses `crossOffOnMain` off the "primary" entry only; find-free-times.ts:136-145 passes the moved booking there and is the only caller with a booking to ignore (find-booking-choices.ts:77 never crossed one off, unchanged). The test with the same hour in primary and Work leaves Work busy. No new defect.

Independent review

The second, passing receipt, over the whole delta after the fixes. The first review, at `88292e5` , passed with F-407 to F-409, fixed in `5ba8870` .

Status: passed **Target commit:** 5ba8870a37850ebfbcf506517eaaf1df07d25b22 **Base commit:** d4cc8837041288c0f84a013ee3f05ecb1505a76b **Base ref:** main **Spec hash:** fffdf6c6a6d10126c8de733b2cba81d451a9ff129080ac42f58737ccb080f64b **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-12T19:30:00Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh

subagent **Actual execution:** automatic **Reviewed at:** 2026-10-12T19:33:00Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete `d4cc8837041288c0f84a013ee3f05ecb1505a76b..5ba8870a37850ebfbcf506517eaaf1df07d25b22` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `git rev-parse HEAD`, `git merge-base main HEAD`, `sha256sum blueprint/context/current-feature.md`, `git status --porcelain`: pass (HEAD, merge base and spec hash match the request; only review.md differs, plus the owner's untracked note outside the review)
- `npm run test --workspace=@scheduleads-app/shared`: pass (188 tests)
- `npm run test --workspace=backend`: pass (88 files, 1013 tests, local Postgres)
- `npm run build --workspace=frontend`: pass (`/settings/calendar` built; tree unchanged after)
- `npm run lint --workspace=frontend`: pass
- `/check`: not run (not required)

Evidence

- Freshness: HEAD 5ba8870, merge base with `main` d4cc883, spec SHA-256 fffdf6c6...f64b, no tracked change but review.md.
- Security: both new routes sit under `/calendar/*` with the dashboard CORS, CSRF and no-store middleware (backend/app.ts:28) plus organization, subscription and `booking` module middleware; the business and person come from the session (calendar-routes.ts GET /calendars, PUT /busy-calendars); the save's update is bound to connection id, organization, person and account email (save-busy-calendars.ts:33-41); the schema is strict, at most 50 unique ids of 1 to 1024 characters; calendar names reach the page as React text; no tokens in answers or logs.
- Contracts: the 409/503 codes and the 400 `field: "calendarIds"` match the spec's Data / contracts; the callback lands on `/settings/calendar`; the migration adds `busyCalendarIds text[] not null default '{primary}'`; a same-account reconnect keeps the pick (case-insensitive SQL compare) and another account resets it.
- F-407: closed. Pre-12h and lapsed connections now get separate messages, both asserted in calendar-routes.test.ts.
- F-408: closed. get-busy-times.ts returns [] for an empty pick before any token call; tested with a needs_reconnect row and an expired key.

- F-409: closed. findBusyBlocks returns per-calendar blocks; the moved booking is crossed off "primary" only; tested with the same hour in primary and Work.
- Tests lens: every spec test rule has a test; no `.only`, `.skip` or todo in the changed test files; Google is faked at `fetch` with call counts asserted.

Findings

- F-410 [P3] open: nothing ticked plus a lapsed connection is told "customers cannot book you", though they can
- F-411 [P3] open: a calendar hidden from the list at Google shows as gone and cannot be re-ticked
- F-407, F-408, F-409: closed this pass

Remaining risk

- `/check` was not run (not required); no browser walkthrough of `/settings/calendar` in this review.
- The real Google round trip (consent with the list permission, the calendar list, free/busy over several calendars, how Google keys "primary" in the answer) is untested against a live account; moved to 10b by the owner (spec Amendments).
- listCalendars' paging (nextPageToken, ten pages) has no test; only matters past 250 calendars.